

Apostila de Python para Data Science

Guia completo de referência — by Bernardo

Material de consulta cobrindo tudo o que estudamos: dos fundamentos da linguagem até manipulação e visualização de dados. Use como referência sempre que esquecer algo.

PARTE 1 — FUNDAMENTOS DA LINGUAGEM

1.1 Variáveis

Uma variável é um nome que guarda um valor na memória. Python tem **tipagem dinâmica** — você não declara o tipo, ele descobre sozinho.

```
nome = "Bernardo"    # texto
idade = 21           # inteiro
altura = 1.75         # decimal
estudando = True      # booleano
```

1.2 Tipos de dados fundamentais

Tipo	O que é	Exemplo
int	número inteiro	42, -7, 2026
float	número decimal	3.14, 8.5
str	texto (entre aspas)	"Olá", 'Python'
bool	verdadeiro ou falso	True, False
None	ausência de valor	None

```
# verificar o tipo de algo
print(type(idade))    # <class 'int'>
```

1.3 Operações matemáticas

```
10 + 3    # 13    soma
10 - 3    # 7     subtração
10 * 3    # 30    multiplicação
10 / 3    # 3.333... divisão (sempre retorna float)
10 // 3   # 3     divisão inteira (descarta o decimal)
10 % 3    # 1     resto da divisão (módulo)
2 ** 8    # 256   potência
```

Dica útil: o operador % (módulo) é ótimo para descobrir se um número é par: `numero % 2 == 0`.

1.4 Operações com texto

```
nome = "Bernardo"
sobrenome = "Carmo"

# concatenação
completo = nome + " " + sobrenome    # "Bernardo Carmo"

# métodos úteis de string
texto = "  Olá Mundo  "
texto.strip()           # "Olá Mundo" (remove espaços das pontas)
texto.lower()           # "  olá mundo  "
texto.upper()           # "  OLÁ MUNDO  "
texto.replace("Olá", "Oi")  # substitui
"a,b,c".split(",")       # ["a", "b", "c"] (separa em lista)
"1".isdigit()           # True (é um dígito?)
```

1.5 f-strings (formatação de texto)

A forma moderna e limpa de inserir variáveis em texto:

```
nome = "Bernardo"
idade = 21

print(f"Olá, {nome}! Você tem {idade} anos.")

# formatando casas decimais (sem alterar o valor real)
valor = 10 / 3
print(f"{valor:.2f}")    # 3.33 (2 casas)
print(f"{valor:.0f}")    # 3 (zero casas)
```

1.6 Conversão de tipos

```
int("42")           # 42 (string para inteiro)
float("3.14")        # 3.14
str(100)             # "100"
int(3.99)            # 3 (trunca, não arredonda)
```

1.7 Input do usuário

`input()` SEMPRE retorna string — converta se precisar de número:

```
nome = input("Digite seu nome: ")
idade = int(input("Digite sua idade: "))    # converte para inteiro
altura = float(input("Digite sua altura: ")) # converte para decimal

# capturar vários números de uma vez
numeros = [float(n) for n in input("Números: ").split()]
```

PARTE 2 — ESTRUTURAS DE DADOS

2.1 Listas

Coleção ordenada e mutável de valores.

```

habilidades = ["Python", "SQL", "Git"]

# acessar por índice (começa em 0)
habilidades[0]      # "Python"
habilidades[-1]     # "Git" (último)

# modificar
habilidades.append("Docker")    # adiciona ao final
habilidades.remove("SQL")       # remove um valor
habilidades[0] = "Python 3"     # altera

# informações
len(habilidades)      # tamanho
"Python" in habilidades # True (verifica se existe)

# fatiar (slicing)
numeros = [10, 20, 30, 40, 50]
numeros[1:3]          # [20, 30] (do índice 1 ao 2)
numeros[:2]           # [10, 20] (do início ao índice 1)
numeros[2:]           # [30, 40, 50] (do índice 2 ao fim)

```

2.2 Dicionários

Coleção de pares **chave: valor**. Fundamental em Data Science.

```

candidato = {
    "nome": "Bernardo",
    "idade": 21,
    "aprovado": True,
    "habilidades": ["Python", "SQL"]
}

# acessar
candidato["nome"]          # "Bernardo"

# adicionar / modificar
candidato["empresa"] = "XP Inc"
candidato["idade"] = 22

# percorrer
for chave, valor in candidato.items():
    print(f"{chave}: {valor}")

```

2.3 Tuplas e conjuntos (resumo)

```

# tupla – como lista, mas imutável (não muda)
coordenadas = (10, 20)

# conjunto (set) – valores únicos, sem ordem
tags = {"python", "sql", "python"} # vira {"python", "sql"}

```

PARTE 3 — CONTROLE DE FLUXO

3.1 Condicionais (if / elif / else)

```

nota = 7.5

```

```
if nota >= 7:
    print("Aprovado")
elif nota >= 5:
    print("Recuperação")
else:
    print("Reprovado")
```

A indentação (espaço antes do código) é obrigatória — é assim que Python sabe o que pertence ao bloco. Use 4 espaços.

Operadores de comparação

```
==    # igual
!=    # diferente
>     # maior
<     # menor
>=    # maior ou igual
<=    # menor ou igual
```

Operadores lógicos

```
and    # E    – os dois precisam ser verdadeiros
or     # OU   – basta um ser verdadeiro
not    # NÃO  – inverte

if idade >= 18 and tem_carteira:
    print("Pode dirigir")
```

3.2 Loop for

Percorre uma sequência.

```
# percorrer uma lista
for habilidade in ["Python", "SQL", "Git"]:
    print(habilidade)

# range – gera sequência de números
for i in range(5):          # 0, 1, 2, 3, 4
    print(i)

range(1, 6)                # 1, 2, 3, 4, 5
range(0, 10, 2)            # 0, 2, 4, 6, 8 (de 2 em 2)
range(10, 0, -1)          # 10, 9, 8... 1 (decrescente)
```

3.3 Loop while

Repete enquanto uma condição for verdadeira.

```
contador = 0
while contador < 3:
    print(contador)
    contador += 1          # CUIDADO: sem isso, loop infinito
```

3.4 break e continue

```
for n in range(10):
    if n == 5:
        break              # PARA o loop completamente
```

```
if n % 2 == 0:
    continue    # PULA para a próxima iteração
print(n)
```

PARTE 4 — FUNÇÕES

4.1 Definindo e chamando

```
def saudacao(nome):
    print(f"Olá, {nome}!")

saudacao("Bernardo")
```

4.2 Retorno de valores

A diferença crucial entre `print` e `return`:

- `print` → mostra na tela, o valor se perde
- `return` → devolve o valor para ser usado depois

```
def calcular_media(notas):
    return sum(notas) / len(notas)

resultado = calcular_media([8, 7, 9])    # guarda o valor
print(resultado)                        # 8.0
```

4.3 Parâmetros com valor padrão

```
def apresentar(nome, cargo="Estagiário"):
    return f"{nome} – {cargo}"

apresentar("Bernardo")                # "Bernardo – Estagiário"
apresentar("Ana", "Data Scientist")   # "Ana – Data Scientist"
```

4.4 Boa prática

Funções devem **processar e retornar** — quem decide o que fazer com o resultado (imprimir, salvar) é o código de fora. Isso torna a função reutilizável.

```
# BOM: função calcula, código externo exibe
def filtrar_aprovados(candidatos):
    return [c for c in candidatos if c["nota"] >= 7]

aprovados = filtrar_aprovados(lista)
print(aprovados)
```

PARTE 5 — NUMPY

NumPy trabalha com arrays numéricos de forma extremamente rápida (operações vetorizadas).

```
import numpy as np

# criar arrays
a = np.array([1, 2, 3, 4, 5])

# operações vetorizadas (aplicam em TODOS de uma vez, sem loop)
a * 2      # [2, 4, 6, 8, 10]
a + 10     # [11, 12, 13, 14, 15]

# estatísticas
a.mean()   # média
a.std()    # desvio padrão
a.sum()    # soma
a.max()    # máximo
a.min()    # mínimo

# arrays multidimensionais (matrizes)
matriz = np.array([[1, 2, 3], [4, 5, 6]])
matriz.shape  # (2, 3) – 2 linhas, 3 colunas

# números aleatórios
np.random.rand(5)          # 5 números entre 0 e 1
np.random.normal(35, 8, 200) # 200 números, média 35, desvio 8
```

PARTE 6 — PANDAS

A biblioteca central de Data Science. Trabalha com **DataFrames** (tabelas).

6.1 Criando estruturas

```
import pandas as pd

# Series – uma coluna
notas = pd.Series([8.5, 7.0, 9.0])

# DataFrame – tabela completa
dados = {
    "nome": ["Ana", "Bruno", "Carla"],
    "idade": [25, 30, 22],
    "nota": [8.5, 5.0, 9.0]
}
df = pd.DataFrame(dados)
```

6.2 Lendo e salvando arquivos

```
# ler
df = pd.read_csv("arquivo.csv")
df = pd.read_excel("arquivo.xlsx")

# salvar
df.to_csv("saida.csv", index=False)
df.to_excel("saida.xlsx", index=False) # precisa de: conda install openpyxl
```

6.3 Explorando os dados (primeira olhada)

```
df.head()      # 5 primeiras linhas
df.tail()      # 5 últimas
```

```
df.sample(5)          # 5 aleatórias
df.shape              # (linhas, colunas)
df.columns            # nomes das colunas
df.dtypes             # tipo de cada coluna
df.info()             # tipos + valores não-nulos
df.describe()         # estatísticas das colunas numéricas
df["col"].unique()    # valores únicos
df["col"].value_counts() # conta cada valor único
```

6.4 Selecionando colunas

```
df["nome"]            # uma coluna (Series)
df[["nome", "nota"]]  # várias colunas (DataFrame)
```

6.5 Filtrando linhas

```
# condição única
df[df["nota"] >= 7]

# múltiplas condições – cada uma entre parênteses
df[(df["nota"] >= 7) & (df["idade"] < 30)] # & = E
df[(df["dept"] == "TI") | (df["dept"] == "RH")] # | = OU
```

6.6 Criando e modificando colunas

```
# operação direta entre colunas (vetorizada)
df["media"] = (df["nota1"] + df["nota2"]) / 2

# lógica condicional por linha (apply + lambda)
df["status"] = df["nota"].apply(lambda x: "Aprovado" if x >= 7 else "Reprovado")
```

6.7 GroupBy — agrupar e agregar

A operação mais valiosa em análise de dados.

```
# média de uma coluna por grupo
df.groupby("departamento")["salario"].mean()

# várias estatísticas de uma vez
df.groupby("departamento")["salario"].agg(["mean", "median", "min", "max",
"count"])

# agregação nomeada (mais limpo)
df.groupby("departamento").agg(
    total=("id", "count"),
    salario_medio=("salario", "mean"),
    idade_media=("idade", "mean")
)

# truque: média de coluna 0/1 = proporção
df.groupby("dept")["saiu"].mean() # taxa de saída por departamento
```

6.8 Ordenando

```
df.sort_values("nota", ascending=False) # decrescente
df.sort_values(["dept", "nota"])        # por múltiplas colunas
```

6.9 Tratando valores nulos

```
df.isnull().sum()          # conta nulos por coluna

df.dropna()                # remove linhas com qualquer nulo
df.dropna(subset=["salario"]) # remove só onde 'salario' é nulo

df["idade"] = df["idade"].fillna(df["idade"].median()) # preenche com mediana
df["dept"] = df["dept"].fillna(df["dept"].mode()[0])  # preenche com moda
```

6.10 Duplicatas

```
df.duplicated().sum()      # conta duplicatas
df.drop_duplicates()       # remove
```

6.11 Padronizando texto (antes de mapear)

```
df["col"] = df["col"].str.strip().str.title() # remove espaços, padroniza
maiúsculas

# mapear categorias para números (quando há ordem)
df["estagio"] = df["estagio"].map({"Baixo": 1, "Médio": 2, "Alto": 3})
```

6.12 Merge (juntar tabelas — como JOIN do SQL)

```
# inner join (só linhas que casam nas duas)
pd.merge(df1, df2, on="id")

# left join (mantém todos da esquerda)
pd.merge(df1, df2, on="id", how="left")
```

6.13 Correlação

```
df.corr()          # matriz de correlação (todas numéricas)
df[["age", "nota"]].corr() # correlação entre colunas específicas
```

PARTE 7 — VISUALIZAÇÃO (Matplotlib + Seaborn)

7.1 Setup

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
```

7.2 Estrutura básica de qualquer gráfico

```
plt.figure(figsize=(10, 5)) # tamanho
sns.histplot(data=df, x="idade") # o gráfico em si
plt.title("Título")          # título
plt.xlabel("Eixo X")          # rótulo X
plt.ylabel("Eixo Y")          # rótulo Y
plt.tight_layout()            # ajusta para não cortar
```



```
plt.savefig("grafico.png", dpi=300, bbox_inches="tight") # salva (ANTES do
show)
plt.show() # exhibe
```

7.3 Os gráficos e quando usar cada um

Gráfico	Quando usar	Função Seaborn
Barras	comparar categorias	sns.barplot, sns.countplot
Histograma	distribuição de 1 variável numérica	sns.histplot
Boxplot	distribuição + outliers	sns.boxplot
Scatter	relação entre 2 numéricas	sns.scatterplot
Heatmap	correlações / matrizes	sns.heatmap
Linha	evolução no tempo	sns.lineplot

7.4 Exemplos práticos

```
# CONTAGEM (quantos de cada categoria)
sns.countplot(data=df, x="categoria", palette="Set2")

# HISTOGRAMA (distribuição) – bins controla nº de faixas, kde adiciona curva
sns.histplot(data=df, x="idade", bins=30, kde=True)

# BOXPLOT (distribuição + outliers)
sns.boxplot(data=df, y="salario")

# BOXPLOT cruzado com alvo (análise bivariada)
sns.boxplot(data=df, x="sobreviveu", y="idade")

# SCATTER (relação entre duas variáveis, hue colore por uma terceira)
sns.scatterplot(data=df, x="idade", y="salario", hue="sobreviveu")

# COUNTPLOT cruzado (hue separa por categoria)
sns.countplot(data=df, x="hora_extra", hue="saiu")

# HEATMAP de correlação
sns.heatmap(df.corr(), annot=True, cmap="coolwarm", center=0, fmt=".2f")
```

Regra de ouro do hue: só use quando a cor codifica uma variável COM significado. Colorir cada barra diferente sem motivo é só poluição visual.

7.5 Painel com vários gráficos

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

sns.countplot(data=df, x="alvo", ax=axes[0, 0])
axes[0, 0].set_title("Gráfico 1")

sns.boxplot(data=df, x="alvo", y="idade", ax=axes[0, 1])
axes[0, 1].set_title("Gráfico 2")

# ... axes[1, 0] e axes[1, 1]
```

```
plt.tight_layout()
plt.show()
```

PARTE 8 — FLUXO DE TRABALHO E FERRAMENTAS

8.1 Ambiente (conda)

```
conda activate estudos-ds # ativa o ambiente
conda deactivate          # desativa
conda install pandas numpy # instala bibliotecas
```

8.2 Git (versionamento)

```
git status # ver o que mudou
git add . # adiciona tudo à staging
git commit -m "mensagem" # cria o snapshot
git push # envia para o GitHub
git pull # baixa do GitHub
git log --oneline # histórico
```

8.3 Comandos de terminal úteis

```
cd pasta # entrar na pasta
cd .. # voltar uma pasta
ls # listar arquivos (dir no CMD)
pwd # mostrar pasta atual
code . # abrir VS Code na pasta atual
python arq.py # rodar um script
mkdir nome # criar pasta
```

APÊNDICE — ERROS COMUNS E COMO EVITAR

Erro	Causa	Solução
<code>input()</code> retorna texto	<code>input</code> sempre é string	converta com <code>int()</code> ou <code>float()</code>
<code>.split</code> não funciona	esqueceu os parênteses	use <code>.split()</code> com <code>()</code>
iterar string com <code>split</code>	<code>.split()</code> separa por espaço	itere direto na string: <code>for c in texto</code>
média igual em todas as linhas	usou <code>.mean()</code> ao criar coluna	use operação direta: <code>(c1 + c2) / 2</code>
<code>return</code> dentro do loop	encerra a função cedo demais	acumule numa lista, retorne no fim
<code>ModuleNotFoundError: openpyxl</code>	falta biblioteca de Excel	<code>conda install openpyxl</code>
loop infinito no <code>while</code>	condição nunca vira <code>False</code>	garanta que a variável muda

Erro	Causa	Solução
		dentro do loop

Lembre-se: ninguém decora tudo isso. Até profissionais sêniores consultam documentação toda hora. O importante é entender O QUE cada coisa faz e saber onde procurar a sintaxe. Esta apostila é seu ponto de partida.